# AI-Assisted Legacy Modernization

Pilot report and proposal: a COBOL mainframe application translated to Rust with Claude Code

Illia Burov

November 2026

# In one slide

**What we did**

- Took a complete mainframe application: a credit-card system in **COBOL**, with online screens, nightly batch jobs and mainframe data files
- Had an AI coding agent, **Claude Code**, translate it into **Rust**, a modern, safe language
- Required **proof**: old and new programs run on the same data and must give the same output

**What we got**

- All **30 programs** translated, working and tested
- **40 of 40** batch outputs and **338 of 338** screens identical to the original
- **41 of 42** test cases passed, the last one's automated part too
- **15 defects** found in the original and documented
- About **3 hours** of AI working time, one engineer supervising

**Recommendation: run a second pilot on one of our own legacy systems, with the same method.**

# Why legacy code is a business risk

**The situation**

- Core business rules live in decades-old COBOL, PL/I or RPG code
- The people who wrote it are retiring; few new engineers learn these languages
- Mainframe capacity and licences are a large, fixed cost
- Manual rewrites are slow and risky: rules are buried in the code, and small differences in rounding, dates or edge cases break the business

**What a good solution must do**

1. Understand the old code, *including its quirks*
2. Produce maintainable code in a modern language
3. **Prove** the new system behaves like the old one
4. Keep people in control of scope, review and release
5. Be fast and repeatable across many systems

# The original application: what CardDemo is for

**CardDemo** is the core system of a **credit-card issuer**, the kind a bank runs on its mainframe: who the customers are, which cards and accounts they hold, what they spent and what they owe.

**Who uses it**

- **Bank staff**, through green-screen terminals: accounts, cards, transactions, bill payments, reports
- **Administrators**: which staff may sign on, and with which role
- **The nightly batch**: unattended programs that process the day's card activity

**What it holds**

- Customers, their accounts and credit cards
- Balances, credit and cash limits, billing-cycle totals
- Every card transaction: purchases, returns, payments, interest
- Transaction types, categories, interest-rate groups
- Staff user ids and roles

# A day in the life of the system

**During the day: online screens**

- **Sign on**: menu by role
- **Accounts**: view, change limits and address
- **Cards**: list, view, block, edit
- **Transactions**: browse, view, add
- **Bill payment**: pay a balance in full
- **Reports**: request for any period
- **Users**: add, change, delete staff

**At night: the batch cycle**

1. **Posting**: check, post or reject the day’s transactions
2. **Interest**: per account and category
3. **Combine** daily and interest transactions
4. **Statements**: per customer, text and web
5. **Reports**: page, account and grand totals
6. **Export / import** for another branch

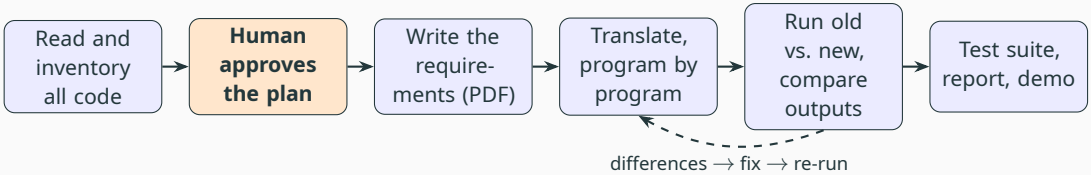
# What was translated

| | Original (COBOL, mainframe) | Translation (Rust) |
|---|---|---|
| Online screens | 17 programs (IBM CICS), 17 screen layouts | 17 modules + terminal screen engine |
| Nightly batch | 13 programs + JCL job scripts | 13 modules + job runner |
| Data | VSAM files, EBCDIC encoding, packed decimals | Embedded database, same record layouts and bytes |
| Size | ≈ 27,000 lines | ≈ 10,800 lines (incl. generated code) |

The application is **AWS CardDemo**, a public sample built to represent real mainframe banking systems. The original code is unchanged in the repository (`cobol/`), next to the translation (`rust/`).

CardDemo © Amazon.com, Inc. or its affiliates, used under the Apache License 2.0. The Rust translation is a derived work by Illia Burov. This pilot is not affiliated with or endorsed by Amazon Web Services.

# How the AI worked: plan first, prove at the end




- Before writing code, the AI asked what to include. A person chose the scope: base application, terminal screens, embedded database, the original's behaviour kept exactly.
- Every phase ended in a saved, reviewable step (25 commits), with a written log of decisions.
- The AI also produced the documents people need: requirements, test suite, demo guide, this report.

# Results at a glance

## Measured results

| Measure | Result |
|---|---|
| Programs translated | 30 / 30 |
| Batch outputs vs. COBOL | 40 / 40 identical |
| Screens vs. COBOL | 338 / 338 identical |
| Manual test cases | 41 / 42 + 1 (step 2) |
| Defects found in the original | 15 |
| AI working time | ≈ 3.1 h |

## Documents produced by the AI

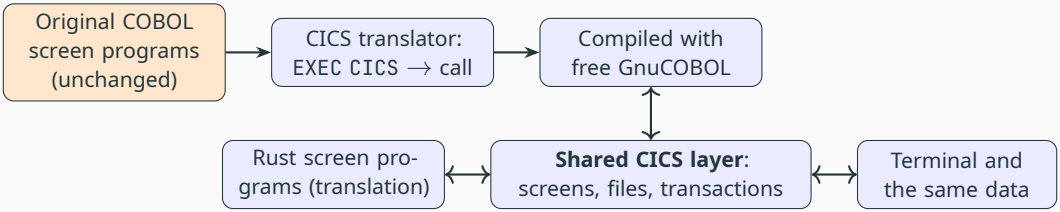
- Requirements specification: 26 pages, 165 functional requirements
- Manual test suite with execution log
- Live demo guide for non-technical audiences
- Progress report with time and token figures
- `CLAUDE.md`: the rulebook for future AI work on this code

# Proof: the old and new systems side by side

| Nightly job | What is compared | Result |
|---|---|---|
| Daily posting | updated accounts, balances, transactions, rejected items | ✓ identical |
| Interest calculation | accounts, generated interest transactions | ✓ identical |
| Transaction report | 519-line printed report | ✓ identical |
| Customer statements | 50 statements, text and web version (7,900 lines) | ✓ identical |
| Data export / import | migration file, re-imported files, error file | ✓ identical |
| File print utilities | 5 programs, incl. packed-decimal outputs | ✓ identical |
| **Online screens** | 338 screens of 49 scripted terminal sessions: every field, colour, cursor | ✓ identical |

Original COBOL (free compiler) and Rust run on **the same data, clock and keystrokes**; one command each repeats the checks.

# The original online screens, live




- Real CICS runs only on licensed mainframes: the original programs get a **stand-in for CICS**, the layer the Rust version already uses. Both runs share it, so the comparison checks the business logic
- All 17 original screen programs run **unchanged**; old and new screens can be shown **side by side**
- Automated comparison: **338 of 338 screens identical**; on the way it found 13 differences in the Rust port (all fixed) and 3 more defects of the original (documented)

# Why not run it on an emulated mainframe?

**The idea**

**Hercules**, a free PC emulator of IBM mainframe hardware: why not run the original on it?

**What a free setup can run**

- MVS 3.8j, the operating system from 1981 that may be used freely
- No CICS: only the hobbyist look-alike **KICKS**
- Only a 1970s COBOL compiler (ANSI COBOL 74)

**Why that does not work for CardDemo**

- CardDemo is modern **Enterprise COBOL** (COBOL 85 and later: EVALUATE, END-IF, inline PERFORM, intrinsic functions); it does not compile with a COBOL 74 compiler
- Making it compile would mean **rewriting the original**, the opposite of what the demo must show
- Current z/OS with real CICS on a PC needs IBM's **licensed** test environment (ZD&T)

**The CICS bridge runs the original code unchanged, at no licence cost.**

# What the AI found that people had missed

**In the original system**

Kept and documented as *known defects*:

- Last account's interest is never saved
- The report counts the last amount twice
- Card update wipes the security code
- Account update moves the group id
- Two programs do not compile
- … 15 in total

**In its own translation**

Found by running the original, and fixed:

- Batch comparison: 9 subtle differences
- AI **code review** (scoped by `CLAUDE.md`): 10 issues
- Online comparison: 13 differences
- All fixed; none was visible in normal tests

**The value is not only in writing code fast, but in verifying it automatically.**

# CLAUDE.md: the project rulebook for the AI

**What it is**

A plain-text file in the repository, read by Claude Code at the start of **every** session:

- **Scope**: what may change
- **How to build, test, prove**
- **Review policy**
- **Lessons learned**

**Excerpt**

```
## Code review
Review only rust/.
- cobol/ is the original source and
  the specification: never report
  findings against it.
- A difference from the COBOL is a
  Rust bug. Known defects (KD-nn)
  are kept on purpose.
```

**One reviewed, versioned rulebook keeps every session consistent.**

# Human oversight built into the workflow

**Control points**

- **Plan approval**: no code until a person accepted plan and scope
- **Checkpoints**: every step is a commit that can be reviewed or undone; nothing is published unasked
- **Budget limits**: the AI stopped at 85% of the usage quota and wrote a hand-over note; work resumed exactly there the next day
- **Memory and logs**: decisions and findings are written down

**Claude Code Remote Control**

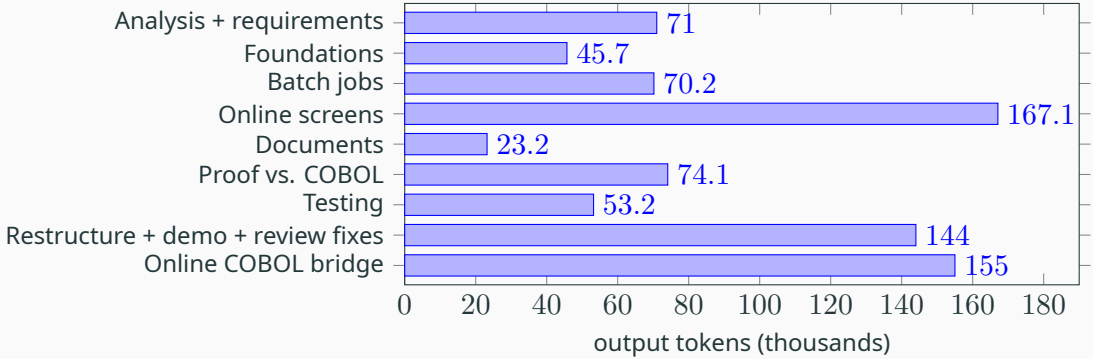
- The work runs on the company workstation; the code stays there
- From the **Claude mobile app**, the engineer follows the session, answers the AI's questions and approves steps while away from the desk
- Long runs no longer need someone sitting at the screen

# Time and tokens

| Session | Work | AI time | Output tokens |
|---|---|---|---|
| 1 | Analysis, requirements, translation of all 30 programs, test suite, report | 1.1 h | 367k |
| 2 | Proof against the original COBOL, test execution, fixes | 0.5 h | 137k |
| 3 | Restructure, demo tooling and guide, `CLAUDE.md`, code-review fixes, this presentation | 0.9 h | 144k |
| 4 | Original COBOL screens on a CICS stand-in, online comparison, fixes | 0.6 h | 155k |
| **Total** | | ≈ **3.1 h** | ≈ **0.8M** |

- Measured from the Claude Code session logs (pauses over 10 minutes removed).
- Token cost depends on the subscription or API price list in use.
- The engineer’s own time went into decisions and reviews, not into typing code.

# Where the effort went




The online screens were the largest part: 17 interactive programs with many validation rules each.

# Risks and mitigations

| Risk | Mitigation in this method |
|---|---|
| AI produces plausible but wrong code | Run old and new on the same data; outputs must be identical |
| Hidden rules in old code are lost | Requirements are written first, from the code, and reviewed |
| Original defects get “fixed” silently | Defects are kept and documented; fixing them is a business decision |
| Parts that cannot run off the mainframe (online CICS screens) | Screens drawn from the original layouts; 41-case test suite; next: comparison on a test mainframe |
| Source code confidentiality | Code stays on company machines; only what is sent to the model leaves them, under the company’s agreement with the vendor |
| Over-reliance on one tool | Output is ordinary Rust code plus documents; any team can maintain it |

# From pilot to programme

**Next pilot (4–6 weeks)**

1. Pick one of our legacy systems: bounded, with test data
2. Write its `CLAUDE.md`: scope, target stack, review rules
3. Set up the comparison harness: old vs. new on the same data
4. Translate, verify, document, as in this pilot
5. Review with the system owners; agree go-live criteria

**What we need**

- One engineer who knows the system, part time
- Access to the code and representative test data
- Claude Code licences for the team
- A test environment for the original system

**Success criteria**

Identical outputs on agreed test data, test suite passed, owners sign off.

# Live demo

```
python3 demo/demo.py cobol      # the original COBOL programs
python3 demo/demo.py rust       # the Rust translation
python3 demo/demo.py compare    # → IDENTICAL, file by file
python3 demo/demo.py online-cobol    # original screens; online = Rust
python3 demo/demo.py online-compare  # → 338/338 screens identical
```

Step-by-step guide for presenters and non-technical audiences: `docs/DEMO.md`.

**Questions: Illia Burov**

# Appendix

# Technical architecture

| Component | Replaces | Lines |
|---|---|---|
| `cobol` | COBOL data rules: decimals, EBCDIC; records generated from copybooks | 1,677 |
| `vsam` | VSAM indexed files, on SQLite | 556 |
| `cics` | CICS screens (from the original BMS files), transfers, file control | 833 |
| `online` | the 17 online programs, one function per COBOL paragraph | 4,283 |
| `batch` | the 13 batch programs and JCL jobs | 1,886 |
| `carddemo` | the application: terminal screens, job runner | 440 |

Verification: `rust/difftest/run.py` (GnuCOBOL comparison) and `cargo test` (21 suites, including the 41 manual cases as scripted terminal sessions and the multi-user tests).

# Known defects of the original (selection)

| ID | Where | Effect |
|---|---|---|
| KD-01 | Interest calculation | the last account’s interest is never written back |
| KD-02/03 | Transaction report | last amount counted twice; no final account total |
| KD-07 | Card update screen | the card security code is wiped |
| KD-12 | Export / import | the programs do not compile as shipped |
| KD-13 | Account update screen | the group id is saved into the ZIP-code field |
| KD-14 | Admin menu | after a not-installed option, the next input is ignored |
| KD-15 | Account update screen | an account with a blank group never counts as unchanged |

All 15 are in the requirements specification, with the exact behaviour the translation reproduces.